

Design and Analysis of Computer Algorithms HW3

Solution Version

Course: Design and Analysis of Computer Algorithms

Exam: Homework 3

Source: HW3.pdf

Year: 2026

1. Find a tight lower-bound class for the problem of finding two closest numbers among n real numbers, $x_1, x_2, \dots, x_n$. (Hint: Find a problem with a known lower-bound and use reduction.)

Reference Answer

Lower bound. We reduce *Element Uniqueness* to Closest Numbers.

Define $\delta := \min_{i \neq j} |x_i - x_j|$. Element Uniqueness asks whether all input numbers are distinct. The key observation is

$$\text{all } x_i \text{ distinct} \iff \delta > 0.$$

Equivalently, using the Vandermonde product,

$$V(x_1, \dots, x_n) := \prod_{1 \leq i < j \leq n} (x_j - x_i),$$

we have

$$V \neq 0 \iff \delta > 0.$$

Given an algorithm for Closest Numbers, run it on the same input. If it returns the closest pair (x_i, x_j) , compute $|x_i - x_j|$. This distance is 0 iff the input contains duplicate numbers. Therefore Element Uniqueness can be solved with only $O(1)$ extra work after solving Closest Numbers.

Since Element Uniqueness has lower bound $\Omega(n \lg n)$ in the comparison decision-tree model, Closest Numbers also has lower bound $\Omega(n \lg n)$. ■

2. Recall the definition and property of binomial trees. Furthermore, we define the node at depth k for a binomial tree B_k is the handle of the B_k tree. Prove the following characterization of an B_k tree: Let T be an B_k tree with handle v . There are disjoint trees $T_0, T_1, \dots, T_{k-1}$, not containing v , with roots $r_0, r_1, \dots, r_{k-1}$, respectively such that
- (1) T_i is an B_i tree, $0 \leq i \leq k-1$, and
 - (2) T results from attaching v to r_0 , and r_i to r_{i+1} , for $0 \leq i < k-1$

Reference Answer

- (1) **Base case $k = 1$.** The tree B_1 consists of a root r_0 and one child v , and v is the handle. Let T_0 be the one-node tree consisting of r_0 . Then T_0 is a B_0 tree, and T is obtained by attaching v to r_0 . Hence both (1) and (2) hold.

Inductive step. Assume the statement is true for B_{k-1} . By the recursive definition of binomial trees, B_k is obtained by linking two disjoint copies of B_{k-1} . The handle v of B_k must lie in the copy that is

attached below the other one, because every node in the upper copy has depth at most $k - 1$, while nodes in the lower copy have their depths increased by 1, so its handle is now at depth k .

Apply the induction hypothesis to that lower B_{k-1} containing v . Then there exist disjoint trees $T_0, T_1, \dots, T_{k-2}$, not containing v , such that T_i is a B_i tree. Let T_{k-1} be the other (upper) copy of B_{k-1} . Since the two copies are disjoint and v lies in the lower copy, T_{k-1} does not contain v . Hence $T_0, T_1, \dots, T_{k-1}$ are disjoint, do not contain v , and satisfy (1). ■

(2) For $k = 1$, statement (2) was proved in the base case above.

For $k > 1$, by the induction hypothesis, inside the lower copy we can build that B_{k-1} by attaching v to r_0 , and r_i to r_{i+1} for $0 \leq i < k - 2$. Since the whole B_k is formed by attaching the root r_{k-2} of this lower B_{k-1} to the root r_{k-1} of the other B_{k-1} , we obtain exactly the chain

$$v \rightarrow r_0 \rightarrow r_1 \rightarrow \dots \rightarrow r_{k-2} \rightarrow r_{k-1},$$

which is equivalent to attaching v to r_0 , and r_i to r_{i+1} for all $0 \leq i < k - 1$. Therefore (2) holds. Thus both statements are true for every k . ■

3. A sequence of n operations is performed on a data structure. The i th operation costs i if i is an exact power of 2, and 1 otherwise. Please determine the amortized cost per operation using

- (a) aggregate analysis
- (b) accounting method of analysis
- (c) potential method of analysis

Reference Answer

(a) Let $m = \lfloor \lg n \rfloor$. The operations at power-of-2 indices are exactly

$$1, 2, 4, \dots, 2^m.$$

Therefore the total cost of n operations is

$$\sum_{j=0}^m 2^j + (n - (m + 1)) = (2^{m+1} - 1) + n - m - 1.$$

Since $2^m \leq n < 2^{m+1}$, we have $2^{m+1} \leq 2n$. Hence

$$\sum_{i=1}^n c_i \leq (2n - 1) + n = 3n - 1.$$

Thus the amortized cost per operation is at most $\frac{3n-1}{n} < 3$, which is $O(1)$.

(b) Charge an amortized cost of 3 to every operation.

For operation $i = 1$, the actual cost is 1, so the remaining 2 units are stored as credit.

If operation $i > 1$ is not a power of 2, then its actual cost is 1, so we place the remaining 2 units as credit on the data structure.

Now consider operation $i = 2^j$ for $j \geq 1$. Since the previous power of 2 is 2^{j-1} , the operations

$$2^{j-1} + 1, 2^{j-1} + 2, \dots, 2^j - 1$$

are all non-powers of 2. There are $2^{j-1} - 1$ such operations, so they store

$$2(2^{j-1} - 1) = 2^j - 2$$

units of credit. The actual cost of operation 2^j is 2^j , and its own amortized charge covers 3 units of this cost, so the remaining $2^j - 3$ units must be drawn from stored credit. Since

$$2^j - 2 \geq 2^j - 3,$$

the stored credit is sufficient. For $j = 1$, this interval is empty, but then the actual cost is 2 and the amortized charge 3 already pays for it. Therefore the credit balance never becomes negative.

Hence charging 3 per operation pays for the entire sequence, so the amortized cost per operation is $3 = O(1)$.

(c) Let D_i denote the data structure after the i th operation, and define the potential function

$$\Phi(D_0) = 0, \quad \Phi(D_i) = 2(i - 2^{\lfloor \lg i \rfloor}) \quad (i \geq 1).$$

This potential is always nonnegative, because $i \geq 2^{\lfloor \lg i \rfloor}$.

The amortized cost of the i th operation is

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

If i is not a power of 2, then $\lfloor \lg i \rfloor = \lfloor \lg(i-1) \rfloor$, so

$$\Phi(D_i) - \Phi(D_{i-1}) = 2.$$

Since $c_i = 1$, we get

$$\hat{c}_i = 1 + 2 = 3.$$

If $i = 1$, then

$$\hat{c}_1 = c_1 + \Phi(D_1) - \Phi(D_0) = 1 + 0 - 0 = 1.$$

If $i = 2^j$ for $j \geq 1$, then

$$\Phi(D_i) = 0$$

and

$$\Phi(D_{i-1}) = 2((2^j - 1) - 2^{j-1}) = 2^j - 2.$$

Since $c_i = 2^j$, we have

$$\hat{c}_i = 2^j + 0 - (2^j - 2) = 2.$$

Thus every operation has amortized cost at most 3, so the amortized cost per operation is $O(1)$.

Remark

In CLRS, the three methods differ only in how we prove an upper bound on the total cost.

For **aggregate analysis**, we do not design any extra bookkeeping. We simply compute or bound the total actual cost of a sequence of n operations, and then divide by n .

For **accounting method**, the design question is: how much should each operation be charged? We assign each operation an amortized charge, possibly larger than its actual cost, and store the extra amount as credit. Usually the cheap operations save credit to pay for a later expensive operation. The invariant is that the total credit is always nonnegative.

For **potential method**, the design question is: what potential function should we choose? We choose $\Phi(D_i) \geq 0$ with $\Phi(D_0) = 0$, and define

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Then

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n c_i.$$

In general, the accounting and potential methods are equivalent at the theorem level. Once an amortized-cost scheme $\hat{c}_i$ is fixed, its cumulative excess charge

$$\sum_{t=1}^i (\hat{c}_t - c_t)$$

can be viewed as a potential $\Phi(D_i)$, and conversely any potential function induces amortized costs through $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.

In this problem, part (b) and part (c) use different schemes. Part (b) uses the uniform charge $\hat{c}_i = 3$ for every operation. Part (c) uses the pattern: 3 for non-powers of 2, 2 for powers 2^j with $j \geq 1$, and 1 for $i = 1$.

Thus

$$\Phi(D_i) = 2(i - 2^{\lceil \lg i \rceil})$$

should be interpreted only as the potential for the scheme in part (c), not as the total stored credit in part (b).

The reason this Φ works is that it has a sawtooth shape: right after an expensive operation at a power of 2, it resets to 0; between two consecutive powers of 2, it grows linearly and accumulates just enough credit to cover the next expensive operation.

Thus the usual pipeline is:

1. identify which operations are expensive and how often they occur
2. choose the proof framework: direct summation, credit scheme, or potential function
3. if using accounting, decide how the cheap operations save credit for future expensive ones
4. if using potential, design Φ so that its increases on cheap operations offset the later drop at an expensive one
5. verify the nonnegativity condition and conclude a uniform upper bound

4. Recall the basic operations related to a minimum priority queue include MINIMUM, INSERT, and DELETE-MIN. Suppose that we use a binary search tree T to implement a minimum priority queue PQ . Please describe how the binary search tree T can support these operations. Besides, we would like PQ can provide the DECREASE-KEY (k, k') operation which decreases the original key value k to a new key value k' , where $k' < k$. For each approach, please provide the time and space complexity.

Reference Answer

Assume the priority queue stores distinct keys. Let h be the height of the binary search tree T .

MINIMUM

1. Basic idea. Since an inorder traversal of a BST lists the keys in sorted order, the minimum key is exactly the leftmost node. Therefore we only need to start from the root and keep following left-child pointers until no further left child exists.

2. Algorithm.

```

BST-Minimum(x):
1  while left(x) ≠ NIL:
2      x ← left(x)
3  return x

```

3. Correctness.**Loop invariant:**

$$\min(T) \in \text{subtree}(x).$$

Initialization: Initially, $x = \text{root}(T)$, so $\text{subtree}(x) = T$. Hence the invariant holds.

Maintenance: Suppose the invariant holds at the start of an iteration and $\text{left}(x) \neq \text{NIL}$. By the BST property,

$$\text{key}(\text{left}(x)) < \text{key}(x),$$

so

$$\min(\text{subtree}(x)) \in \text{subtree}(\text{left}(x)).$$

After setting $x \leftarrow \text{left}(x)$, the invariant is preserved.

Termination: The loop terminates when $\text{left}(x) = \text{NIL}$. Then every key in $\text{subtree}(\text{right}(x))$ exceeds $\text{key}(x)$. By the invariant, $\min(T) \in \text{subtree}(x)$, so $x = \min(T)$. Therefore the algorithm returns the minimum node of T . ■

4. Time and extra-space complexity. The algorithm follows at most one root-to-leaf path, whose length is at most h . Thus MINIMUM takes $O(h)$ time and $O(1)$ extra space.

INSERT(k)

1. Basic idea. To preserve the BST property, the new key must be placed exactly where a BST search for k would terminate. Starting from the root, compare k with the current key, go left if k is smaller, and go right otherwise. When a NIL child is reached, attach the new node there.

2. Algorithm.

```

BST-Insert( $T, z$ ):
1  let  $y \leftarrow \text{NIL}$ 
2  let  $x \leftarrow \text{root}(T)$ 
3  while  $x \neq \text{NIL}$ :
4       $y \leftarrow x$ 
5      if  $\text{key}(z) < \text{key}(x)$ :
6           $x \leftarrow \text{left}(x)$ 
7      else:
8           $x \leftarrow \text{right}(x)$ 
9   $\text{parent}(z) \leftarrow y$ 
10 if  $y = \text{NIL}$ :
11      $\text{root}(T) \leftarrow z$ 
12 else if  $\text{key}(z) < \text{key}(y)$ :
13      $\text{left}(y) \leftarrow z$ 
14 else:
15      $\text{right}(y) \leftarrow z$ 

```

3. Correctness.

Loop invariant: the correct BST position for $\text{key}(z)$ lies in $\text{subtree}(x)$, and $y = \text{parent}(x)$.

Initialization: Initially, $x = \text{root}(T)$ and $y = \text{NIL}$. Hence the invariant holds.

Maintenance: Suppose the invariant holds at the start of an iteration. By the BST property, exactly one of the two children of x can contain the correct position for $\text{key}(z)$: if $\text{key}(z) < \text{key}(x)$, it must lie in the left subtree; otherwise it must lie in the right subtree. The algorithm moves x to that child and sets $y \leftarrow \text{parent}(x)$, so the invariant is preserved.

Termination: The loop terminates when $x = \text{NIL}$. Then y is the direct parent of the insertion position. Attaching z as the left child of y when $\text{key}(z) < \text{key}(y)$, and as the right child otherwise, preserves BST order. If T was empty, then $y = \text{NIL}$ and z becomes the root. Therefore the resulting tree is a BST containing all old keys plus the new key k . ■

4. Time and extra-space complexity. The algorithm examines only one search path from the root to a leaf position, whose length is at most h . Hence INSERT takes $O(h)$ time and $O(1)$ extra space.

DELETE-MIN

1. Basic idea. First find the minimum node. Since the minimum node has no left child, deleting it is simpler than general BST deletion: we only need to replace it by its right child.

2. Algorithm.

```

BST-Delete-Min(T):
1  let  $x \leftarrow \text{BST-Minimum}(\text{root}(T))$ 
2  if  $x = \text{root}(T)$ :
3       $\text{root}(T) \leftarrow \text{right}(x)$ 
4      else:
5           $\text{left}(\text{parent}(x)) \leftarrow \text{right}(x)$ 
6          if  $\text{right}(x) \neq \text{NIL}$ :
7               $\text{parent}(\text{right}(x)) \leftarrow \text{parent}(x)$ 
8      return  $x$ 

```

3. Correctness. Let

$$x = \min(T), \quad p = \text{parent}(x).$$

Since x is minimum, $\text{left}(x) = \text{NIL}$.

Case 1: $x = \text{root}(T)$. The algorithm sets $\text{root}(T) \leftarrow \text{right}(x)$. The BST order on subtree($\text{right}(x)$) is unchanged, so the remaining tree is still a BST.

Case 2: $x \neq \text{root}(T)$. Then $x = \text{left}(p)$, so

$$\text{key}(x) < \text{key}(p).$$

By the BST property, every key in subtree($\text{right}(x)$) lies in the interval

$$(\text{key}(x), \text{key}(p)).$$

Therefore replacing $\text{left}(p)$ by $\text{right}(x)$ preserves BST order.

In both cases, the remaining tree is a BST with $\min(T)$ removed. Thus the algorithm correctly implements DELETE-MIN. ■

4. Time and extra-space complexity. Finding the minimum takes $O(h)$ time. After that, the pointer updates take $O(1)$ time. Hence DELETE-MIN takes $O(h)$ time and $O(1)$ extra space.

DECREASE-KEY(k, k')

1. Basic idea. We assume $k' < k$ and that the key k is present in T . A first idea is to find the node x of key k and simply replace its key by k' . However, this may violate the BST property. Some node in the left subtree of x may now have key at least k' , or some ancestor a for which x lies in the right subtree of a may satisfy $k' \leq \text{key}(a)$. Thus x may no longer belong to its current position.

Once the key changes, the node must be moved to the position determined by the new key. The clean way to do this is to decompose the operation into two standard BST operations: delete the node of key k , and then insert a node of key k' .

2. Algorithm.

BST-Decrease-Key(T, k, k'):

```

1  let  $x \leftarrow \text{BST-Search}(\text{root}(T), k)$ 
2   $\text{BST-Delete}(T, x)$ 
3  let  $\text{new} \leftarrow \text{new-node}(k')$ 
4   $\text{BST-Insert}(T, \text{new})$ 
5  return  $\text{new}$ 

```

3. Correctness. Let S be the original key set of T . By the distinct-key assumption, BST-Search returns the unique node x with $\text{key}(x) = k$. After $\text{BST-Delete}(T, x)$, the tree is a BST with key set

$$S \setminus \{k\}.$$

After BST-Insert with key k' , the tree is again a BST with key set

$$(S \setminus \{k\}) \cup \{k'\}.$$

Therefore the final tree is exactly the result of decreasing key k to k' . ■

4. Time and extra-space complexity. BST-Search , BST-Delete , and BST-Insert each take $O(h)$ time on a BST of height h . Therefore

$$T_{\text{DECREASE-KEY}} = O(h) + O(h) + O(h) = O(h).$$

The algorithm uses only a constant number of pointers and variables in addition to the tree itself, so the extra space is $O(1)$.

Thus, all four operations take $O(h)$ time and $O(1)$ extra space, where h is the height of T . For an unbalanced BST of n nodes, such as one created by inserting the keys in sorted order, the tree becomes a chain and its height is $n - 1$, so each operation costs $O(n)$. If the BST maintains a balance condition, such as in a red-black tree or an AVL tree, then $h = O(\log n)$, and all four operations run in $O(\log n)$ time.